

Reducción de Dimensionalidad

PCA, Isomap, t-SNE y UMAP aplicados al dataset Wheat Seeds

Material de apoyo — Unidad 2 · Minería de Datos (TUIA, UNR, 2026)

Explicación conceptual y práctica a partir del código de la cátedra

1. Introducción: ¿por qué reducir dimensionalidad?

El dataset Wheat Seeds describe cada semilla con 7 variables numéricas (área, perímetro, compacidad, longitud y ancho del núcleo, coeficiente de asimetría, longitud del surco) y una etiqueta de variedad (Kama, Rosa, Canadian). Un espacio de 7 dimensiones no se puede graficar ni "mirar" directamente: los humanos interpretamos bien hasta 2 o 3 dimensiones a la vez.

Las técnicas de reducción de dimensionalidad buscan proyectar los datos originales a un espacio de menor dimensión (típicamente 2 o 3) intentando conservar la mayor cantidad posible de la "estructura" o información relevante de los datos originales. Sirven principalmente para tres cosas:

- **Visualización exploratoria:** ver si existen agrupamientos naturales (clusters) o si las clases conocidas se separan bien en el espacio reducido.
- **Reducción de ruido y redundancia:** muchas variables están correlacionadas entre sí y aportan información repetida.
- **Preprocesamiento para otros algoritmos:** trabajar con menos dimensiones puede acelerar el entrenamiento de un clasificador o clustering posterior, y a veces mejora resultados al eliminar variables poco informativas.

En el script, 'category' se usa únicamente para **colorear** los gráficos: en ningún momento se entrena un modelo supervisado. Es un análisis **no supervisado** de exploración visual — se busca ver si, sin decirle al algoritmo cuál es la variedad de cada semilla, la estructura geométrica de los datos ya alcanza para separarlas.

1.1. El paso previo obligatorio: estandarización

Antes de aplicar cualquiera de las cuatro técnicas, el código aplica **StandardScaler**, que transforma cada variable con la fórmula $z = (x - \text{media}) / \text{desvío estándar}$, dejando cada columna con media 0 y varianza 1.

¿Por qué es indispensable acá? Las cuatro técnicas de este script (PCA, Isomap, t-SNE y UMAP) se basan, en mayor o menor medida, en **distancias** o en la **varianza** entre puntos. Si una variable se mide en una escala mucho más grande que otra (por ejemplo, área en mm² puede tomar valores de cientos, mientras que la compacidad es un número entre 0 y 1), esa variable dominaría por completo el cálculo de distancias o de varianza, no porque sea más informativa, sino simplemente por su escala numérica. Estandarizar pone a todas las variables en un pie de igualdad antes de que el algoritmo decida qué direcciones o vecindades son relevantes.

Cómo debería usarse en la práctica: el escalado se debe calcular (fit) siempre sobre el conjunto de entrenamiento, y luego aplicarse (transform) igual a validación/test, para no filtrar información del conjunto de evaluación hacia el ajuste del escalador (data leakage). En este script, al ser un análisis exploratorio sobre todo el dataset, se aplica fit_transform sobre el total — algo aceptable acá, pero que en un pipeline de modelado con train/test sí requiere separar los pasos.

2. PCA — Análisis de Componentes Principales

Qué es: PCA es un método **lineal** de reducción de dimensionalidad. Busca nuevos ejes (llamados componentes principales), que son combinaciones lineales de las variables originales, ordenados de manera que el primer eje (PC1) capture la mayor varianza posible de los datos, el segundo (PC2) capture la mayor varianza posible entre las direcciones ortogonales (perpendiculares) a PC1, y así

sucesivamente.

Por qué funciona así: la intuición es que la varianza es una forma de medir "cuánta información" aporta una dirección: si todos los puntos tuvieran el mismo valor en una dirección, esa dirección no distinguiría nada entre las muestras. Matemáticamente, PCA se obtiene calculando los autovectores y autovalores de la matriz de covarianza de los datos (o, de forma equivalente y más estable numéricamente, mediante la descomposición SVD (Singular Value Decomposition), que es lo que usa internamente scikit-learn — por eso el objeto PCA expone también `singular_values_`).

Parámetros y atributos usados en el código

Elemento	Qué hace
<code>n_components=3</code>	Cantidad de componentes a conservar (de 7 originales a 3).
<code>fit_transform()</code>	Calcula las direcciones principales (fit) y proyecta los datos sobre ellas (transform), en un solo paso.
<code>explained_variance_</code>	Varianza (en unidades absolutas) que captura cada componente — indica cuánta información retiene cada eje nuevo.
<code>singular_values_</code>	Valores singulares de la descomposición SVD subyacente; relacionados matemáticamente con <code>explained_variance_</code> .

El scree plot: el gráfico de barras (varianza explicada por componente) junto con la curva de varianza acumulada (`np.cumsum`) es la herramienta estándar para decidir **cuántas componentes conservar**. En la práctica se busca el "codo" de la curva de barras (el punto donde agregar una componente más deja de aportar mucha varianza extra), o directamente un umbral sobre la curva acumulada, por ejemplo quedarse con las componentes necesarias para explicar el 90-95% de la varianza total.

Cuándo conviene usar PCA: cuando se sospecha que las relaciones entre variables son mayormente lineales, cuando se necesita rapidez (es el método más liviano computacionalmente de los cuatro) y reproducibilidad exacta (no depende de aleatoriedad), o cuando se quiere interpretar qué variables originales pesan más en cada componente (mediante `components_`).

Limitación clave: al ser lineal, PCA no puede "desenrollar" estructuras curvas (por ejemplo, datos distribuidos en forma de espiral o de "S"). Si la separación real entre clases depende de relaciones no lineales entre variables, PCA puede mostrar clases superpuestas aunque en realidad sean separables.

3. Isomap

Qué es: Isomap (Isometric Mapping) es un método **no lineal**, pensado para datos que viven sobre una **variedad (manifold)** curva dentro de un espacio de mayor dimensión — el ejemplo clásico es el "Swiss roll": una hoja curvada en forma de rollo, donde la distancia en línea recta entre dos puntos puede ser engañosa si esos puntos están en capas distintas del rollo, aunque estén "cerca" siguiendo la superficie.

Cómo funciona: el algoritmo sigue tres pasos: (1) construye un grafo de vecinos, conectando cada punto con sus `k` vecinos más cercanos (parámetro `n_neighbors`); (2) calcula la **distancia geodésica** entre todo par de puntos como el camino más corto sobre ese grafo (no la distancia en línea recta),

usando un algoritmo tipo Dijkstra/Floyd-Warshall; (3) aplica un método similar a PCA clásico (MDS, Multidimensional Scaling) sobre esa matriz de distancias geodésicas para encontrar una proyección de baja dimensión que las preserve lo mejor posible.

Parámetros usados: `n_neighbors=6` define el tamaño del vecindario para construir el grafo (un valor muy chico puede partir el grafo en componentes desconectadas; uno muy grande empieza a comportarse como PCA, perdiendo la ventaja de capturar la curvatura local). `n_components=2` fija la dimensión de salida.

Cuándo conviene usarlo: cuando hay razones para pensar que los datos tienen una estructura curva de baja dimensión intrínseca (por ejemplo, datos de imágenes de un objeto rotando, donde el "verdadero" grado de libertad es el ángulo de rotación). Es más costoso computacionalmente que PCA porque requiere calcular distancias entre todos los pares de puntos.

Limitación: es sensible al parámetro `n_neighbors` y a outliers, que pueden crear "atajos" falsos en el grafo de vecinos y distorsionar las distancias geodésicas calculadas.

4. t-SNE

Qué es: t-SNE (t-distributed Stochastic Neighbor Embedding) es una técnica **no lineal** diseñada específicamente para **visualización** en 2D o 3D, no como un método general de reducción de dimensionalidad para otros fines (por ejemplo, no se recomienda usar su salida como entrada de un clasificador posterior).

Cómo funciona (idea general): en el espacio original, calcula para cada par de puntos una probabilidad de que sean "vecinos" (más alta cuanto más cerca están). Luego, en el espacio reducido, intenta encontrar una configuración de puntos donde esas mismas probabilidades de vecindad se preserven, usando una distribución t de Student de colas pesadas (de ahí la "t" del nombre) para las distancias en baja dimensión, lo que ayuda a separar bien clusters distintos y evita que todos los puntos queden amontonados en el centro del gráfico. La optimización se hace de forma iterativa, minimizando la diferencia (divergencia de Kullback-Leibler) entre ambas distribuciones de probabilidad.

Parámetros usados en el código:

- **perplexity=40:** controla, de forma aproximada, cuántos vecinos "efectivos" se consideran relevantes para cada punto. Es el hiperparámetro más importante de t-SNE: valores bajos (5-10) priorizan estructura muy local (pueden fragmentar clusters reales en varios pedacitos); valores altos (30-50) consideran vecindarios más amplios. No hay un valor "correcto" universal — se recomienda probar varios y comparar.
- **n_iter=300:** número de iteraciones del optimizador. Muy pocas iteraciones pueden dar una proyección que no llegó a converger (estructura todavía inestable).
- **verbose=1:** solo controla si se imprime el progreso en consola, no afecta el resultado.

Por qué se mide el tiempo de ejecución: t-SNE tiene una complejidad computacional que crece fuertemente con la cantidad de muestras (en su versión clásica, de forma cuadrática), por lo que en datasets grandes puede ser considerablemente más lento que PCA o UMAP. El script mide el tiempo justamente para evidenciar este costo.

Punto clave para interpretar el gráfico: las **distancias entre clusters** en un gráfico de t-SNE, y el **tamaño** de cada cluster, no son directamente interpretables ni comparables entre sí (dos clusters muy separados en

el gráfico no necesariamente están más "lejos" en el espacio original que dos clusters más próximos). Lo único que t-SNE intenta preservar de forma confiable es qué puntos son vecinos locales de cuáles.

5. UMAP

Qué es: UMAP (Uniform Manifold Approximation and Projection) es también una técnica **no lineal**, conceptualmente emparentada con t-SNE e Isomap (se apoya en la idea de manifold y en preservar vecindades), pero basada en fundamentos de topología y teoría de grafos, con una implementación más eficiente.

Cómo se diferencia de t-SNE: UMAP construye también un grafo de vecindad (similar en espíritu al de Isomap), pero optimiza directamente una función de similitud entre grafos (usando entropía cruzada difusa) en lugar de la divergencia KL de t-SNE. En la práctica, esto trae dos ventajas frecuentes: (1) suele ser notablemente más **rápido**, escalando mejor a datasets grandes; y (2) tiende a preservar **algo más de estructura global** además de la local — las distancias relativas entre clusters en UMAP suelen ser algo más informativas que en t-SNE, aunque tampoco deben sobre-interpretarse.

Parámetros usados en el código: `init='random'` define cómo se inicializan las posiciones antes de optimizar (una alternativa común es `init='spectral'`, que suele dar convergencia más estable, aunque más lenta de calcular). `random_state=0` fija la semilla para que el resultado sea reproducible entre corridas.

Detalle a corregir/tener en cuenta: en el código, UMAP se aplica sobre `xWheat` (sin escalar) en vez de `xWheatScaled`, a diferencia de PCA, Isomap y t-SNE. Por la misma razón explicada en la sección de preprocesamiento (sensibilidad a la escala de las variables), lo recomendable para mantener consistencia metodológica sería usar también `xWheatScaled` acá.

Cuándo conviene usarlo: es en general una buena opción "por defecto" hoy en día para exploración visual de datasets medianos/grandes: combina velocidad, buena preservación de clusters locales y algo de estructura global, con menos sensibilidad a hiperparámetros que t-SNE (aunque también tiene los suyos: `n_neighbors` y `min_dist`, no usados explícitamente en este script porque se dejaron en su valor por defecto).

6. Cuadro comparativo

Técnica	Tipo	Preserva	Velocidad	Uso típico
PCA	Lineal	Varianza global	Muy alta	Preprocesamiento, interpretación de variables, primer vistazo rápido
Isomap	No lineal	Distancias geodésicas (global)	Media/baja	Datos con estructura de manifold curva conocida
t-SNE	No lineal	Vecindad local	Baja	Visualización fina de clusters en datasets chicos/medianos
UMAP	No lineal	Local + algo de global	Alta	Exploración visual general, datasets grandes

7. De la práctica a la teoría: ideas para relacionar con la materia

Este script conecta directamente con varios ejes del programa de Minería de Datos:

- **Preparación y normalización de datos:** el uso de StandardScaler es un caso concreto de por qué la preparación de datos es un paso previo obligatorio antes de aplicar casi cualquier algoritmo basado en distancias.
- **Aprendizaje no supervisado:** las cuatro técnicas no usan la etiqueta 'category' para ajustar el modelo, solo se la usa después, para colorear y evaluar visualmente qué tan bien la estructura descubierta coincide con las clases reales.
- **Relación con clustering:** una buena proyección 2D/3D suele ser un paso previo útil antes de aplicar un algoritmo de clustering (ej. k-means), tanto para visualizar los resultados como, a veces, para aplicar el clustering directamente sobre el espacio reducido.
- **Selección de hiperparámetros:** el scree plot de PCA y la elección de perplexity en t-SNE o n_neighbors en Isomap/UMAP son ejemplos de decisiones de diseño que conviene justificar y probar empíricamente, algo que suele pedirse al defender un TP.

Para preparar la defensa oral de un TP que use estas técnicas, conviene poder responder con seguridad: ¿por qué elegiste esta técnica y no otra?, ¿qué significa el eje X/Y del gráfico?, ¿qué pasaría si cambiás el hiperparámetro principal?, y ¿por qué fue necesario escalar los datos antes de aplicar el método?